Properties in Spring Boot

Application Properties

- Spring Boot loads configuration from `src/main/resources/application.yml`.
- **YAML** (YAML Ain't Markup Language) provides a **structured, human-readable** way to define hierarchical configurations.

```
server:
  port: 7000

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/mydb
    username: user
    password: pass

logging:
  level:
    root: INFO
    com:
      example: DEBUG
```

Common Application Properties

1️⃣ Server Configuration

```
server:
  port: 7000
```

2️⃣ Datasource Configuration (PostgreSQL)

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/mydb
    username: myuser
    password: mypass
```

3️⃣ Logging Configuration

```
logging:
  level:
    root: INFO
    com:
      example: DEBUG
```

Full list of common Spring Boot properties → Documentation

Custom Properties

- Allow defining **application-specific settings**
- Loaded from external configuration files (e.g., `application.yml`)
- Injected into beans via `@Value` → **decouples configuration from code**

Defining Custom Properties

```
server:
  port: 7000

spring:
  main:
    banner-mode: "off"

app:
  name: My Application
  version: 1.0.0
  max-users: 1000
```

Injecting Custom Properties with `@Value`

```
@Component
public class AppService {

    private final String appName;
    private final String appVersion;
    private final Integer maxUsers;

    public AppService(
            @Value("${app.name}") String appName,
            @Value("${app.version}") String appVersion,
            @Value("${app.max-users}") Integer maxUsers) {
        this.appName = appName;
        this.appVersion = appVersion;
        this.maxUsers = maxUsers;
    }

    @PostConstruct
    public void printAppDetails() {
        System.out.println("Application Name: " + appName);
        System.out.println("Version: " + appVersion);
        System.out.println("Max Users: " + maxUsers);
    }
}
```

Profiles

- Profiles allow **environment-specific configuration**: development, testing, staging, production.
- Override default properties based on the **active profile**.

```
# Default profile
server:
  port: 7000

app:
  name: My Application
  version: 1.0.0
  max-users: 1000

---
# Docker profile
spring.config.activate.on-profile: docker
server:
  port: 8080

---
# No-banner profile
spring.config.activate.on-profile: no-banner
spring:
  main:
    banner-mode: "off"
```

Activating Profiles

1️⃣ In `application.yml`

```
spring:
  profiles:
    active: docker
```

2️⃣ Command-Line Arguments

```
mvn clean package -Dmaven.skip.test=true

java -jar target/properties-0.0.1-SNAPSHOT.jar --spring.profiles.active=docker,no-banner
```

3️⃣ Environment Variables

```
SPRING_PROFILES_ACTIVE=docker,no-banner java -jar target/properties-0.0.1-SNAPSHOT.jar
```

4️⃣ Docker Compose

```
services:
  properties:
    build: .
    environment:
      - SPRING_PROFILES_ACTIVE=docker,no-banner
```

```
mvn clean package -Dmaven.test.skip=true
docker compose up --build -d
```

5️⃣ Programmatically

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication app = new SpringApplication(App.class);
        app.setAdditionalProfiles("dev");
        app.run(args);
    }
}
```

Key Takeaways

- **`application.yml`** centralizes Spring Boot configuration
- Custom properties → inject via `@Value`
- Profiles → environment-specific overrides
- PostgreSQL example shown for datasource configuration

Resources

- Properties with Spring and Spring Boot
- Spring Profiles